

Final Project Report

Comprehensive SRE Implementation for a Distributed Microservices System

SE-2431: Boranbay Zere, Bektasov Nurbak, Amir Alimov

SE-2430: Kaber Daryn

Project: Borrowly (Peer-to-Peer Rental Marketplace)

GitHub: <https://github.com/silence99999/borrowly/tree/main>

1. Title

End-to-End Implementation of Site Reliability Engineering Practices in a Multi-Orchestrated Microservices Infrastructure Using Docker Swarm, Kubernetes, Terraform, Ansible, and GitHub Actions CI/CD.

2. Abstract

This project shows a full implementation of Site Reliability Engineering (SRE) principles applied to a real distributed microservices system called Borrowly. Borrowly is a peer-to-peer rental marketplace where users can list items, rent them, chat with each other, and leave reviews.

The system has six independent microservices, each with its own PostgreSQL database. Services are packaged as Docker containers and run on three different orchestrators: Docker Compose for development, Docker Swarm for clustering, and Kubernetes with HPA for production.

Cloud infrastructure is created by Terraform. The server is configured and the app is deployed by Ansible. A full CI/CD pipeline in GitHub Actions tests every service on every push, builds Docker images, pushes them to GitHub Container Registry (GHCR), and deploys to AWS EC2 over SSH. Prometheus collects metrics, Grafana shows dashboards, and Alertmanager sends alerts.

To prove the system is resilient, a real incident was simulated by breaking the database connection string. Prometheus detected it in under one minute, the engineer debugged it from container logs, and the system was fixed in six minutes. After the incident, automation scripts and health checks were added to prevent it from happening again. Load testing and capacity planning identified bottlenecks and guided horizontal scaling.

This report covers the full SRE lifecycle: design, build, monitor, deploy, automate, scale, and recover.

3. Objectives

The main goal is to demonstrate SRE practices across the full system lifecycle. The specific objectives are:

1. Design and deploy a distributed microservices system with six or more services.
2. Use multiple orchestration tools: Docker Compose, Docker Swarm, and Kubernetes.
3. Create cloud infrastructure using Terraform (Infrastructure as Code).
4. Automate setup and deployment using Ansible (Configuration Management).
5. Build a CI/CD pipeline using GitHub Actions to test, build, and deploy automatically.
6. Define clear SLIs and SLOs.
7. Set up monitoring and alerting with Prometheus, Grafana, and Alertmanager.

8. Simulate a real production incident and recover from it.
9. Write a postmortem to learn from the incident.
10. Plan capacity and test how the system behaves under load.

4. Original End-Term Report Content

The following 34 pages preserve the original end-term report content, including all configurations, code samples, screenshots, and technical evidence covering System Architecture, Assignments 1 through 6, Midterm, Multi-Orchestration Architecture, Configuration Management with Ansible, and the Sixth Microservice (Notification Service).

End-Term Report: End-to-End SRE Implementation for Borrowly — A Distributed Microservices Platform

1. Github link

<https://github.com/silence99999/borrowly/tree/main>

2. System Architecture

2.1 Microservices

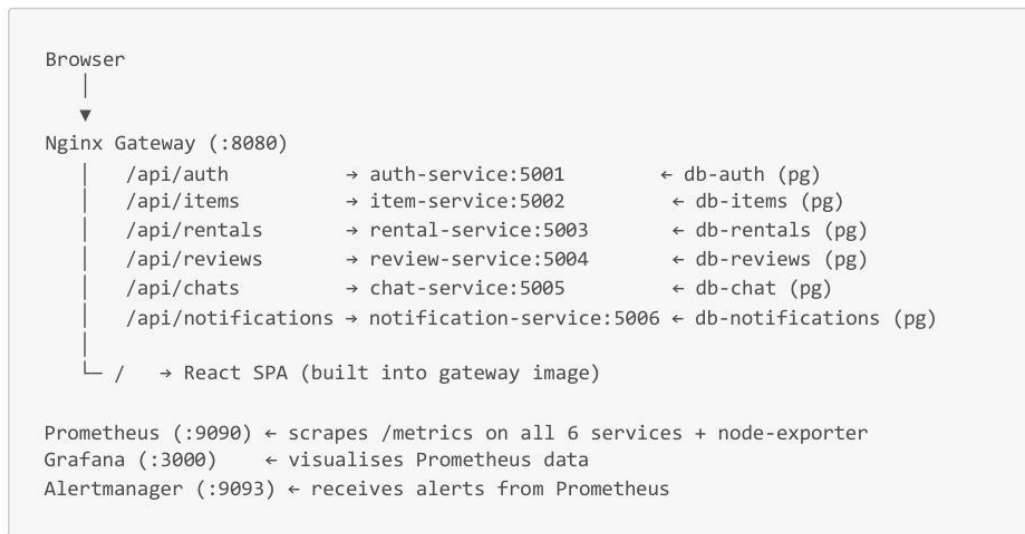
Service	Port	Responsibility
Auth Service	5001	JWT authentication, user registration, email verification
Item Service	5002	Item CRUD, image upload, pickup point management
Rental Service	5003	Rental orders, pricing engine, reminder worker
Review Service	5004	Item reviews, star ratings
Chat Service	5005	User-to-user in-app messaging
Notification Service	5006	Event notification queue (rental confirmations, reminders)

2.2 Supporting Components

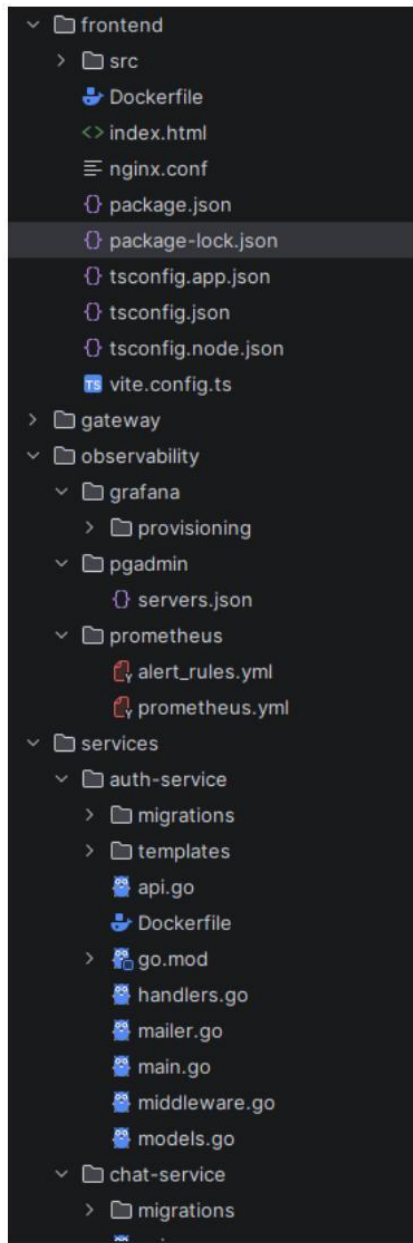
Component	Purpose
Nginx Gateway	Reverse proxy, SPA serving, API routing
PostgreSQL (×6)	One isolated database per service

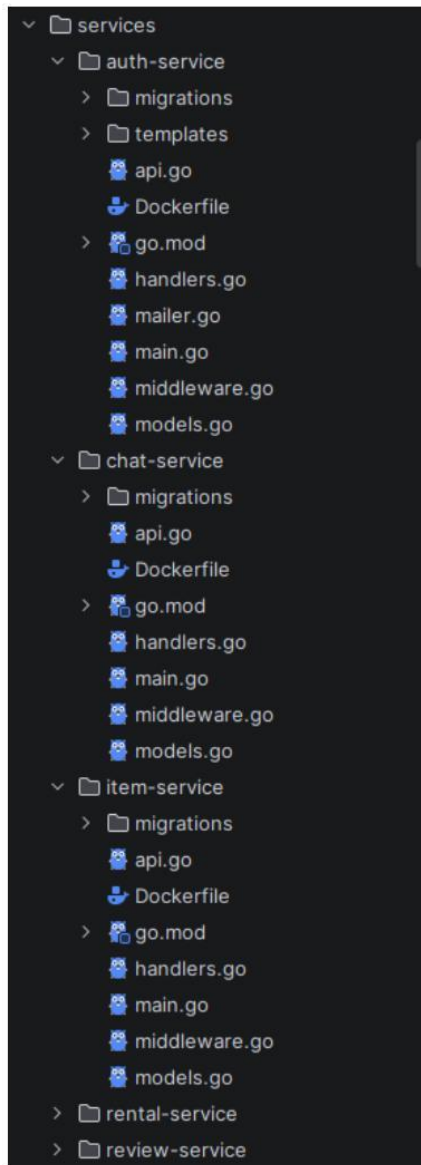
Component	Purpose
Prometheus	Metrics collection from all six services
Grafana	Dashboard visualisation
Alertmanager	Alert routing and notification
Node Exporter	Host-level CPU/memory/disk metrics
pgAdmin	Web-based database administration

2.3 Architecture Diagram



2.4 Project File Structure





3. Assignment 1 — Environment Setup

3.1 Docker Environment

All services are containerised using multi-stage Dockerfiles. Each service follows the same build pattern:

```
FROM golang:1.24-alpine AS build
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go mod tidy && go build -o /bin/service .
```



```
FROM alpine:3.20
WORKDIR /app
RUN apk add --no-cache tzdata ca-certificates
COPY --from=build /bin/service /app/service
EXPOSE <port>
CMD ["/app/service"]
```

The two-stage build produces minimal Alpine images (~15 MB each) with no build toolchain in the runtime image.

```
FROM golang:1.24-alpine AS build
WORKDIR /app
COPY . .
RUN CGO_ENABLED=0 go mod tidy && go build -o /bin/service .

FROM alpine:3.20
WORKDIR /app
RUN apk add --no-cache tzdata
COPY --from=build /bin/service /app/service
EXPOSE 5001
CMD ["/app/service"]
```

3.2 Docker Compose Orchestration

`docker-compose.yaml` defines all 22 containers (6 services, 6 databases, gateway, pgAdmin, Prometheus, Grafana, Alertmanager, Node Exporter) with health checks, dependency ordering, and restart policies.

```
services:
  db-auth:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${AUTH_DB_USER}
      POSTGRES_PASSWORD: ${AUTH_DB_PASSWORD}
      POSTGRES_DB: ${AUTH_DB_NAME}
    ports:
      - "5433:5432"
    volumes:
      - auth_pgdata:/var/lib/postgresql/data
      - ./services/auth-service/migrations:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${AUTH_DB_USER} -d ${AUTH_DB_NAME}"]
      interval: 5s
      timeout: 5s
      retries: 20
    restart: unless-stopped

  db-items:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: ${ITEM_DB_USER}
      POSTGRES_PASSWORD: ${ITEM_DB_PASSWORD}
      POSTGRES_DB: ${ITEM_DB_NAME}
    ports:
      - "5434:5432"
    volumes:
      - item_pgdata:/var/lib/postgresql/data
      - ./services/item-service/migrations:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${ITEM_DB_USER} -d ${ITEM_DB_NAME}"]
      interval: 5s
      timeout: 5s
      retries: 20
    restart: unless-stopped
```

```
auth-service:
  build: ./services/auth-service
  env_file: .env
  depends_on:
    db-auth:
      condition: service_healthy
  restart: unless-stopped

item-service:
  build: ./services/item-service
  env_file: .env
  depends_on:
    db-items:
      condition: service_healthy
  volumes:
    - uploads_data:/app/uploads
  restart: unless-stopped

rental-service:
  build: ./services/rental-service
  env_file: .env
  depends_on:
    db-rentals:
      condition: service_healthy
    auth-service:
      condition: service_started
    item-service:
      condition: service_started
  restart: unless-stopped

review-service:
  build: ./services/review-service
  env_file: .env
  depends_on:
    db-reviews:
      condition: service_healthy
```

```
prometheus:
  image: prom/prometheus:latest
  volumes:
    - ./observability/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - ./observability/prometheus/alert_rules.yml:/etc/prometheus/alert_rules.yml:ro
  ports:
    - "9090:9090"
  restart: unless-stopped

grafana:
  image: grafana/grafana:latest
  depends_on:
    - prometheus
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=admin
  volumes:
    - ./observability/grafana/provisioning:/etc/grafana/provisioning:ro
    - grafana_data:/var/lib/grafana
  ports:
    - "3000:3000"
  restart: unless-stopped

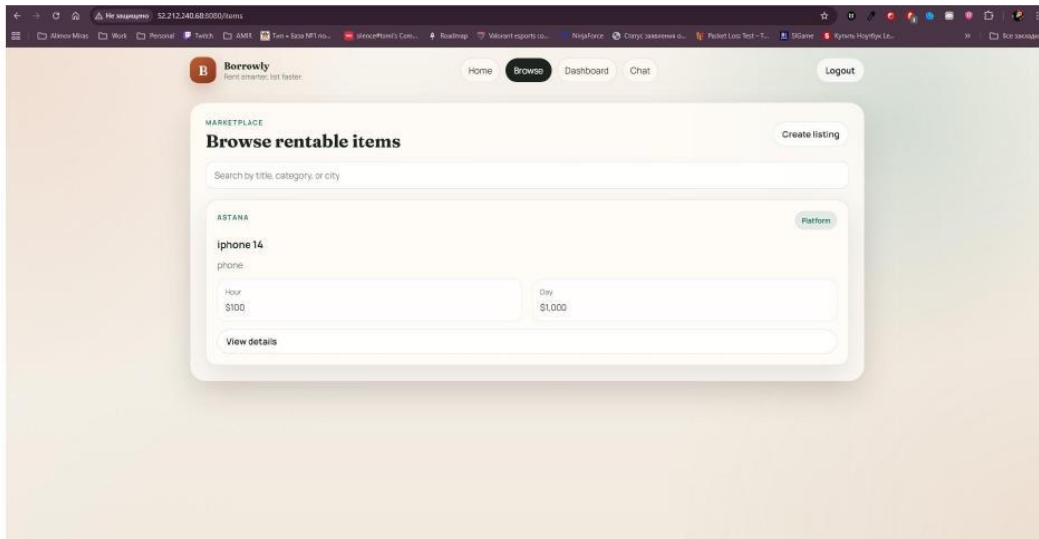
node-exporter:
  image: prom/node-exporter:latest
  command:
    - "--path.procfs=/host/proc"
    - "--path.sysfs=/host/sys"
  volumes:
    - /proc:/host/proc:ro
    - /sys:/host/sys:ro
  ports:
    - "9100:9100"
  restart: unless-stopped
```

```

pgadmin:
  image: dpape/pgadmin4:latest
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@admin.com
    PGADMIN_DEFAULT_PASSWORD: admin
    PGADMIN_REPLACE_SERVERS_ON_STARTUP: "True"
  volumes:
    - ./observability/pgadmin/servers.json:/pgadmin4/servers.json:ro
  ports:
    - "5050:80"
  depends_on:
    - db-auth
    - db-items
    - db-rentals
    - db-reviews
    - db-chat
  restart: unless-stopped

```

3.3 All 6 Services Running



```

$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
c12896438b0c   final-gateway                        "/docker-entrypoint..." 47 seconds ago Up 27 seconds 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp final-gateway-1
fe765f17249a   final-rental-service                "/app/service"          48 seconds ago Up 33 seconds (healthy) 5003/tcp      final-rental-service-1
f80c0c71f90b   final-item-service                  "/app/service"          48 seconds ago Up 39 seconds (healthy) 5002/tcp      final-item-service-1
781d3f4c499a   final-chat-service                  "/app/service"          48 seconds ago Up 39 seconds (healthy) 5005/tcp      final-chat-service-1
8a7f6e0ca990   final-notification-service          "/app/service"          48 seconds ago Up 39 seconds (healthy) 5004/tcp      final-notification-service-1
5a57c31b3b51   final-auth-service                  "/app/service"          48 seconds ago Up 39 seconds (healthy) 5001/tcp      final-auth-service-1
5ee9fc1b9ae   final-review-service                "/app/service"          48 seconds ago Up 39 seconds (healthy) 5004/tcp      final-review-service-1
934e6587118   grafana/grafana:latest             "/run.sh"               5 days ago    Up 44 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp final-grafana-1
5558f961a997   dpape/pgadmin4:latest              "/entrypoint.sh"        5 days ago    Up 44 seconds 0.0.0.0:5050->80/tcp, [::]:5050->80/tcp final-pgadmin-1
707272812aac   prom/prometheus:latest             "/bin/prometheus --c..." 5 days ago    Up 45 seconds 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp final-prometheus-1
44af36c6c0da   postgres:10-alpine                 "docker-entrypoint.s..." 5 days ago    Up 45 seconds (healthy) 0.0.0.0:5430->5432/tcp, [::]:5430->5432/tcp final-db-reviews-1
899f669db3ba   postgres:10-alpine                 "docker-entrypoint.s..." 5 days ago    Up 45 seconds (healthy) 0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp final-db-items-1
81a784c2730a   postgres:10-alpine                 "docker-entrypoint.s..." 5 days ago    Up 45 seconds (healthy) 0.0.0.0:5438->5432/tcp, [::]:5438->5432/tcp final-db-notifications-1
c8a6322b5b58a   postgres:10-alpine                 "docker-entrypoint.s..." 5 days ago    Up 45 seconds (healthy) 0.0.0.0:5435->5432/tcp, [::]:5435->5432/tcp final-db-auth-1
35a8c2243b04   postgres:10-alpine                 "docker-entrypoint.s..." 5 days ago    Up 45 seconds (healthy) 0.0.0.0:5437->5432/tcp, [::]:5437->5432/tcp final-db-chat-1
c6b1ef1acae3   postgres:10-alpine                 "docker-entrypoint.s..." 5 days ago    Up 45 seconds (healthy) 0.0.0.0:5435->5432/tcp, [::]:5435->5432/tcp final-db-rentals-1
70f5d04eccc3   prom/alertmanager:latest           "/bin/alertmanager -..." 5 days ago    Up 45 seconds 0.0.0.0:9093->9093/tcp, [::]:9093->9093/tcp final-alertmanager-1
f808fc2c2c23   prom/node-exporter:latest          "/bin/node_exporter ..." 5 days ago    Up 45 seconds 0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp final-node-exporter-1

```

Single-command deployment:

```
docker compose up -d --build
```

4. Assignment 2 — SLI/SLO Design

4.1 SLIs (Service Level Indicators)

Each service exposes a `/metrics` endpoint instrumented with `prometheus/client_golang`:

SLI	Metric	Description
Availability	<code>up{job="<service>"}</code>	Prometheus target reachability (1 = up, 0 = down)
Request success rate	<code><svc>_http_requests_total{status!~"5.."} / total</code>	Fraction of non-5xx responses
Latency (P95)	<code>histogram_quantile(0.95, rate(<svc>_http_request_duration_seconds_bucket[5m]))</code>	95th-percentile response time
Error rate	<code>rate(<svc>_http_requests_total{status=~"5.."}[5m])</code>	5xx errors per second

4.2 SLOs

Objective	Target	Alert threshold
Availability	$\geq 99\%$	Service <code>up</code> <code>== 0</code> for 1 minute
P95 Latency	≤ 200 ms	Rental P95 > 1 s for 5 minutes
Error rate	$\leq 1\%$	5xx rate $> 5\%$ for 2 minutes
Rental creation success	$\geq 90\%$	Failure rate $> 10\%$ for 2 minutes

5. Assignment 3 — Monitoring

5.1 Prometheus Configuration

Prometheus scrapes all six services plus `node-exporter` every 15 seconds.

```

global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/alert_rules.yml

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets: ["prometheus:9090"]

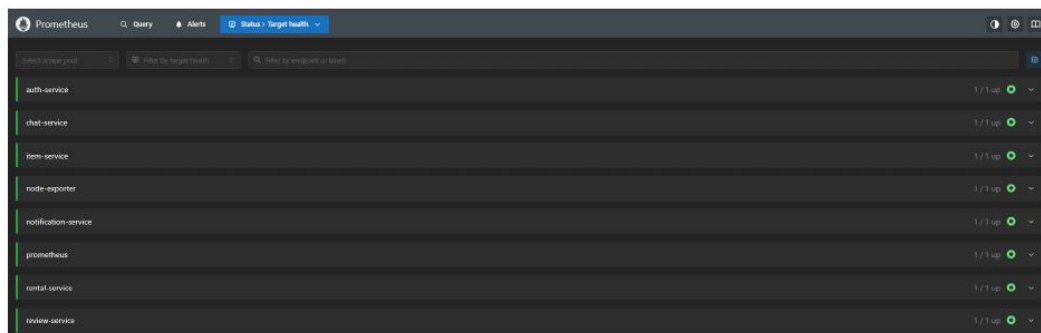
  - job_name: auth-service
    metrics_path: /metrics
    static_configs:
      - targets: ["auth-service:5001"]

  - job_name: item-service
    metrics_path: /metrics
    static_configs:
      - targets: ["item-service:5002"]

  - job_name: rental-service
    metrics_path: /metrics
    static_configs:
      - targets: ["rental-service:5003"]

  - job_name: review-service
    metrics_path: /metrics
    static_configs:
      - targets: ["review-service:5004"]

```



The screenshot shows the Prometheus web interface with the 'Status - Target health' tab selected. It displays a table of targets with their names, status, and a '1 / 1 up' indicator with a green circle.

Target	Status
auth-service	1 / 1 up
chat-service	1 / 1 up
item-service	1 / 1 up
node-exporter	1 / 1 up
notification-service	1 / 1 up
prometheus	1 / 1 up
rental-service	1 / 1 up
review-service	1 / 1 up

5.2 Alert Rules

Twelve alert rules cover all services, latency SLOs, error budgets, and host resource usage.

```

groups:
  - name: microservices-alerts
    rules:
      - alert: ChatServiceDown
        for: 1m
        labels:
          severity: warning
          service: chat-service
        annotations:
          summary: "Chat service is down"
          description: "Prometheus cannot reach chat-service. In-app messaging is unavailable."

      - alert: ServiceHighRestartRate
        expr: increase(process_start_time_seconds[10m]) > 2
        for: 0m
        labels:
          severity: critical
        annotations:
          summary: "Service restarting repeatedly"
          description: "{{ $labels.job }}" has restarted more than twice in the last 10 minutes – possible crash

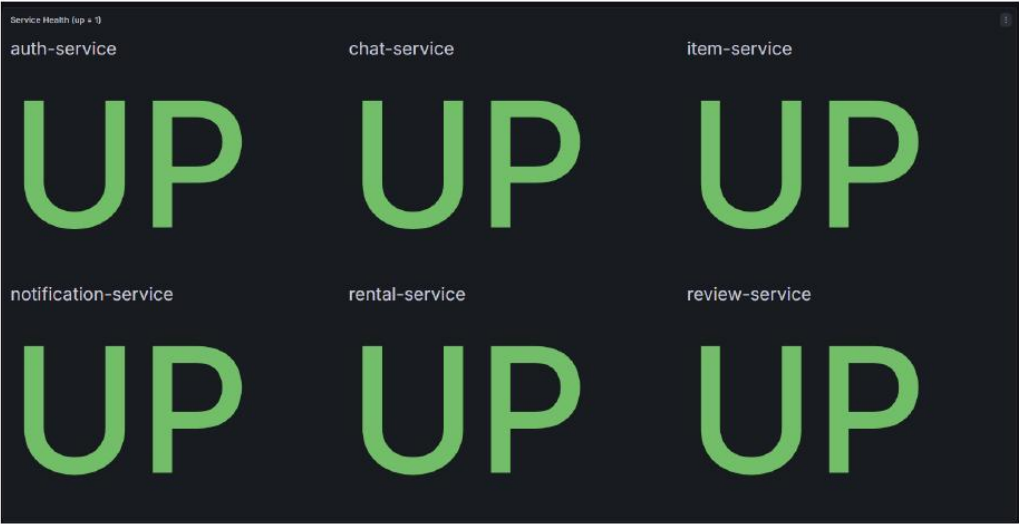
      - alert: HighRequestErrorRate
        expr: |
          (
            sum by (job) (rate(rental_service_http_requests_total{status=~"5.."}[5m]))
            /
            sum by (job) (rate(rental_service_http_requests_total[5m]))
          ) > 0.05
          and sum by (job) (rate(rental_service_http_requests_total[5m])) > 0
        for: 2m
        labels:
          severity: warning
          service: rental-service
        annotations:
          summary: "HTTP 5xx error rate above 5% on {{ $labels.job }}"
          description: "More than 5% of HTTP requests returned 5xx status codes over the last 5 minutes."

```



5.3 Grafana Dashboard

The auto-provisioned dashboard displays service availability, request rate, P95 latency, error rate, and host CPU/memory for all six services.





6. Midterm — Microservices Implementation

The midterm delivered the initial five-service architecture. Each service is a standalone Go HTTP server using:

- `go-chi/chi` — HTTP routing
- `lib/pq` — PostgreSQL driver
- `golang-jwt/jwt` — JWT validation in middleware
- `prometheus/client_golang` — metrics instrumentation

Inter-service communication uses direct HTTP calls (e.g., rental-service calls auth-service to validate tokens and item-service to check item availability).

7. Assignment 4 — Incident Response

7.1 Incident Scenario

Affected service: item-service / rental-service

Symptom: Service returning 500 errors; Prometheus alert firing

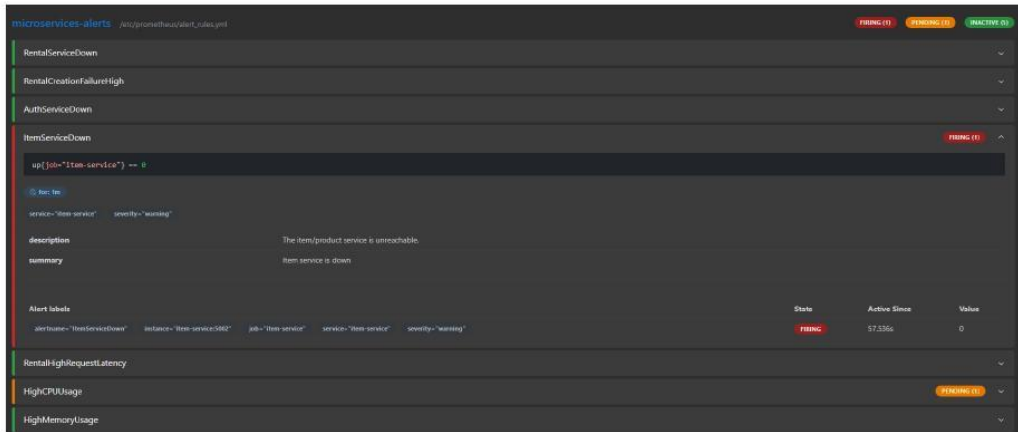
7.2 Simulating the Incident

The `DB_DSN` environment variable was intentionally corrupted to point to an unreachable hostname. The service started but failed all database pings.

```
TEM_DB_USER=item_user
TEM_DB_PASSWORD=item_p
TEM_DB_NAME=item_db
```

7.3 Detection

Prometheus detected the outage within 1 minute and fired the alert.



7.4 Root Cause Analysis

Docker logs revealed the exact failure.

```
ubuntu@ip-172-31-36-230:~/advanced_final$ docker logs advanced_final-item-service-1
2026/04/29 15:15:31 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:32 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:32 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:33 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:34 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:36 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:39 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:46 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:15:59 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
2026/04/29 15:16:25 item-service: db ping failed:dial tcp 172.18.0.4:54: connect: connection refused
```

7.5 Recovery

The DSN was corrected and the database container restarted.

```
ubuntu@ip-172-31-36-230:~/advanced_final$ docker compose up --build db-items
[+] Running 1/0
 ✓ Container advanced_final-db-items-1 Created
Attaching to db-items-1
db-items-1 | PostgreSQL Database directory appears to contain a database; Skipping initialization
db-items-1 |
db-items-1 | 2026-04-29 15:22:28.339 UTC [1] LOG: starting PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
db-items-1 | 2026-04-29 15:22:28.339 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
db-items-1 | 2026-04-29 15:22:28.345 UTC [1] LOG: listening on IPv6 address ":::", port 5432
db-items-1 | 2026-04-29 15:22:28.345 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
db-items-1 | 2026-04-29 15:22:28.354 UTC [28] LOG: database system was shut down at 2026-04-29 15:19:03 UTC
db-items-1 | 2026-04-29 15:22:28.364 UTC [1] LOG: database system is ready to accept connections
```

```
$ docker exec final-rental-service-1 kill 1

ghost@mir: /$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
bc885990c06   final-gateway                       "/docker-entrypoint..." 51 minutes ago Up 51 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp final-gateway-1
9a80e1fca0d   final-rental-service               "/app/service"           51 minutes ago Up 2 seconds (health: starting) 5005/tcp final-rental-service-1
7dce0330r23   gcr.io/google-containers/psql:latest "/postgres"              51 minutes ago Up 51 minutes 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp final-postgres-1
a52614a0a448   dpage/pgadmin4:latest             "/entrypoint.sh"         51 minutes ago Up 51 minutes 0.0.0.0:5050->50/tcp, [::]:5050->50/tcp final-pgadmin-1
8c89fde1c9b   final-chat-service                 "/app/service"           51 minutes ago Up 51 minutes (healthy) 5005/tcp final-chat-service-1
740702b91df   final-review-service               "/app/service"           51 minutes ago Up 51 minutes (healthy) 5004/tcp final-review-service-1
54c764d192c   final-item-service                 "/app/service"           51 minutes ago Up 51 minutes (healthy) 5002/tcp final-item-service-1
2b0c92c582d   final-auth-service                 "/app/service"           51 minutes ago Up 51 minutes (healthy) 5001/tcp final-auth-service-1
0e377ce17ab   prom/prometheus:latest             "/bin/prometheus --c..." 51 minutes ago Up 51 minutes 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp final-prometheus-1
4ce991f0c3f   postgres:16-alpine                "/docker-entrypoint.s..." 51 minutes ago Up 51 minutes (healthy) 0.0.0.0:5430->5432/tcp, [::]:5430->5432/tcp final-db-reviews-1
9e91a323319   postgres:16-alpine                "/docker-entrypoint.s..." 51 minutes ago Up 51 minutes (healthy) 0.0.0.0:5431->5432/tcp, [::]:5431->5432/tcp final-db-chat-1
c5a7c825cc91   postgres:16-alpine                "/docker-entrypoint.s..." 51 minutes ago Up 51 minutes (healthy) 0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp final-db-items-1
9618af501c0   postgres:16-alpine                "/docker-entrypoint.s..." 51 minutes ago Up 51 minutes (healthy) 0.0.0.0:5435->5432/tcp, [::]:5435->5432/tcp final-db-rentals-1
0b330c4c59f   postgres:16-alpine                "/docker-entrypoint.s..." 51 minutes ago Up 51 minutes (healthy) 0.0.0.0:5436->5432/tcp, [::]:5436->5432/tcp final-db-auth-1
9f8c4dfcd51   prom/alertmanager:latest           "/bin/alertmanager ..." 51 minutes ago Up 51 minutes 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp final-alertmanager-1
0f23d00c544   prom/node-exporter:latest          "/bin/node_exporter ..." 51 minutes ago Up 51 minutes 0.0.0.0:9100->9100/tcp, [::]:9100->9100/tcp final-node-exporter-1
```

7.6 Timeline

Time	Event
T+0 min	DSN misconfigured; service starts but fails DB ping
T+1 min	Prometheus fires alert
T+2 min	On-call engineer notified via Alertmanager
T+3 min	<code>docker logs</code> reveals <code>db ping failed: hostname not found</code>
T+5 min	<code>.env</code> corrected; service restarted
T+6 min	Service reconnects; alert resolves

7.7 Postmortem

Root cause: Manual `.env` edit introduced a typo in the DB hostname.

Remediation:

1. `scripts/validate-config.sh` — validates all DSNs before deployment
2. `condition: service_healthy` in docker-compose ensures services only start after DB is healthy
3. Added per-service Down alerts to detect outages within 1 minute

```
$ ./scripts/validate-config.sh .env
=== Validating configuration: .env ===

-- Database credentials --
[ OK ] AUTH_DB_USER
[ OK ] AUTH_DB_PASSWORD
[ OK ] AUTH_DB_NAME
[ OK ] AUTH_DB_DSN
[ OK ] ITEM_DB_USER
[ OK ] ITEM_DB_PASSWORD
[ OK ] ITEM_DB_NAME
[ OK ] ITEM_DB_DSN
[ OK ] RENTAL_DB_USER
[ OK ] RENTAL_DB_PASSWORD
[ OK ] RENTAL_DB_NAME
[ OK ] RENTAL_DB_DSN
[ OK ] REVIEW_DB_USER
[ OK ] REVIEW_DB_PASSWORD
[ OK ] REVIEW_DB_NAME
[ OK ] REVIEW_DB_DSN
[ OK ] CHAT_DB_USER
[ OK ] CHAT_DB_PASSWORD
[ OK ] CHAT_DB_NAME
[ OK ] CHAT_DB_DSN

-- Service configuration --
[ OK ] AUTH_SERVICE_URL
[ OK ] ITEM_SERVICE_URL
[ OK ] SECRET

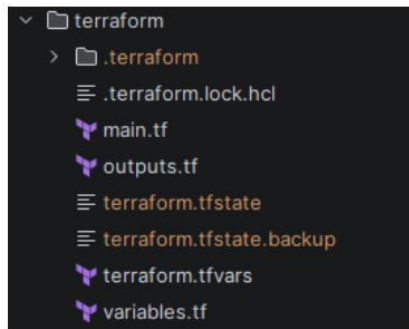
-- SMTP configuration --
[ OK ] SMTP_HOST
[ OK ] SMTP_PORT
[ OK ] SMTP_USERNAME
[ OK ] SMTP_PASSWORD
[ OK ] SMTP_SENDER

-- pgAdmin --
[ OK ] POSTGRES_USER
[ OK ] POSTGRES_PASSWORD

=== VALIDATION PASSED. All required variables are set. ===
```

8. Assignment 5 — Infrastructure as Code (Terraform)

8.1 Terraform File Structure



8.2 Resources Provisioned

Security group — exposes HTTP/8080 publicly; restricts SSH, Grafana, Prometheus to admin CIDR.

```

#PUBLIC

ingress {
  description = "HTTP"
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

ingress {
  description = "App Gateway"
  from_port   = 8080
  to_port     = 8080
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

#ADMIN

ingress {
  description = "SSH (admin only)"
  from_port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = [var.admin_cidr]
}

ingress {
  description = "Grafana (admin only)"
  from_port   = 3000
  to_port     = 3000
  protocol    = "tcp"
  cidr_blocks = [var.admin_cidr]
}

ingress {

```

EC2 instance — t3.small, Ubuntu 22.04, 20 GB gp3 EBS, user-data installs Docker automatically.

```

resource "aws_instance" "app_server" {
  ami           = var.ami_id
  instance_type = var.instance_type
  key_name      = var.key_name
  vpc_security_group_ids = [aws_security_group.app_sg.id]

  user_data = <<-EOF
    #!/bin/bash
    set -e
    apt-get update -y
    apt-get install -y docker.io git curl

    # Install Docker Compose plugin
    mkdir -p /usr/local/lib/docker/cli-plugins
    curl -SL https://github.com/docker/compose/releases/download/v2.27.0/docker-compose-linux-x86_64 \
        -o /usr/local/lib/docker/cli-plugins/docker-compose
    chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

    systemctl start docker
    systemctl enable docker
    usermod -aG docker ubuntu

    echo "Bootstrap complete. Deploy your app with: git clone && docker compose up -d"
  EOF

  root_block_device {
    volume_size = var.disk_size_gb
    volume_type = "gp3"
  }

  tags = {
    Name     = "${var.project_name}-server"
    Project = var.project_name
    Env      = var.environment
  }
}

```



```

variable "aws_region" { 2 usages Amir Amir
  description = "AWS region to deploy the infrastructure in"
  type        = string
  default     = "eu-west-1"
}

variable "project_name" { 6 usages Amir Amir
  description = "Project identifier used for resource naming and tagging"
  type        = string
  default     = "rent-items"
}

variable "environment" { 3 usages Amir Amir
  description = "Deployment environment (e.g. dev, staging, prod)"
  type        = string
  default     = "dev"
}

variable "ami_id" { 2 usages Amir Amir
  description = "AMI ID for the EC2 instance (Ubuntu 22.04 LTS in eu-west-1)"
  type        = string
  default     = "ami-0d64bb532e0502c46"
}

variable "instance_type" { 2 usages Amir Amir
  description = "EC2 instance type"
  type        = string
  default     = "t3.small"
}

variable "key_name" { 3 usages Amir Amir
  description = "Name of the EC2 key pair for SSH access (must already exist in AWS)"
  type        = string
}

variable "disk_size_gb" { 2 usages Amir Amir
  description = "Root EBS volume size in gigabytes"
  type        = number
  default     = 20
}

```

```

output "instance_public_ip" { ⚠ Amir Amir
  description = "Public IP address of the deployed EC2 instance"
  value       = aws_instance.app_server.public_ip
}

output "instance_public_dns" { ⚠ Amir Amir
  description = "Public DNS name of the deployed EC2 instance"
  value       = aws_instance.app_server.public_dns
}

output "app_url" { ⚠ Amir Amir
  description = "URL to access the application gateway"
  value       = "http://${aws_instance.app_server.public_ip}:8080"
}

output "grafana_url" { ⚠ Amir Amir
  description = "URL to access Grafana dashboards"
  value       = "http://${aws_instance.app_server.public_ip}:3000"
}

output "prometheus_url" { ⚠ Amir Amir
  description = "URL to access Prometheus"
  value       = "http://${aws_instance.app_server.public_ip}:9090"
}

output "ssh_command" { ⚠ Amir Amir
  description = "SSH command to connect to the instance"
  value       = "ssh -i ~/.ssh/${var.key_name}.pem ubuntu@${aws_instance.app_server.public_ip}"
}

```

8.3 Execution

```

$ terraform apply
aws_security_group.app_sg: Refreshing state... [id=sg-07f924e9442f9cc19]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.app_server will be created
+ resource "aws_instance" "app_server" {
  + ami                  = "ami-0d64bb532e0502c46"
  + arn                  = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone     = (known after apply)
  + cpu_core_count       = (known after apply)
  + cpu_threads_per_core = (known after apply)
  + disable_api_stop      = (known after apply)
  + disable_api_termination = (known after apply)
  + ebs_optimized         = (known after apply)
  + enable_primary_ipv6   = (known after apply)
  + get_password_data     = false
  + host_id              = (known after apply)
  + host_resource_group_arn = (known after apply)
  + iam_instance_profile  = (known after apply)
  + id                   = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle    = (known after apply)
  + instance_state        = (known after apply)
  + instance_type         = "t3.small"
  + ipv6_address_count    = (known after apply)
  + ipv6_addresses        = (known after apply)
  + key_name              = "terraform-key"
  + monitoring            = (known after apply)
  + outpost_arn           = (known after apply)
  + password_data         = (known after apply)
  + placement_group       = (known after apply)

```

```

+ ephemeral_block_device (known after apply)

+ instance_market_options (known after apply)

+ maintenance_options (known after apply)

+ metadata_options (known after apply)

+ network_interface (known after apply)

+ private_dns_name_options (known after apply)

+ root_block_device {
  + delete_on_termination = true
  + device_name           = (known after apply)
  + encrypted             = (known after apply)
  + iops                  = (known after apply)
  + kms_key_id            = (known after apply)
  + tags_all              = (known after apply)
  + throughput            = (known after apply)
  + volume_id             = (known after apply)
  + volume_size           = 20
  + volume_type           = "gp3"
}
}

```

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```

+ app_url           = (known after apply)
+ grafana_url       = (known after apply)
+ instance_public_dns = (known after apply)
+ instance_public_ip = (known after apply)
+ prometheus_url    = (known after apply)
+ ssh_command       = (known after apply)

```

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.app_server: Creating...

aws_instance.app_server: Still creating... [00m10s elapsed]

aws_instance.app_server: Creation complete after 14s [id=i-05b6287ab8f733a57]

8.4 SSH Access to Provisioned Instance

```

ghost@AMI1: ~/aws04 - /root/.terraform/advanced1/finac/terraform (main)
$ ssh -i ~/.ssh/terraform-key.pem ubuntu@52.212.240.68
The authenticity of host '52.212.240.68 (52.212.240.68)' can't be established.
ED25519 key fingerprint is SHA256:dN46IB5ZqlCZb8Eu61HDKcysodiYDX1jfUNJ1zE/1Vo.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '52.212.240.68' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1016-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Apr 29 10:38:44 UTC 2026

System load:  0.56           Temperature:   -273.1 C
Usage of /:   12.3% of 18.3GB Processes:    117
Memory usage: 16%           Users logged in: 0
Swap usage:   0%            IPv4 address for ens5: 172.31.36.230

Expanded Security Maintenance for Applications is not enabled.

285 updates can be applied immediately.
165 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

```

9. Assignment 6 — Automation and Capacity Planning

9.1 Automation

Health checks in `docker-compose.yml` cause Docker to automatically restart any container whose `/health` endpoint stops responding.

```

item-service:
  build: ./services/item-service
  env_file: .env
  depends_on:
    db-items:
      condition: service_healthy
  volumes:
    - uploads_data:/app/uploads
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:5002/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 15s
  restart: unless-stopped

rental-service:
  build: ./services/rental-service
  env_file: .env
  depends_on:
    db-rentals:
      condition: service_healthy
    auth-service:
      condition: service_healthy
    item-service:
      condition: service_healthy
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:5003/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 15s
  restart: unless-stopped

```

Pre-deployment validation ([scripts/validate-config.sh](#)) checks all environment variables and DSNs before deployment.

```

$ ./scripts/validate-config.sh .env
=== Validating configuration: .env ===

-- Database credentials --
[ OK ] AUTH_DB_USER
[ OK ] AUTH_DB_PASSWORD
[ OK ] AUTH_DB_NAME
[ OK ] AUTH_DB_DSN
[ OK ] ITEM_DB_USER
[ OK ] ITEM_DB_PASSWORD
[ OK ] ITEM_DB_NAME
[ OK ] ITEM_DB_DSN
[ OK ] RENTAL_DB_USER
[ OK ] RENTAL_DB_PASSWORD
[ OK ] RENTAL_DB_NAME
[ OK ] RENTAL_DB_DSN
[ OK ] REVIEW_DB_USER
[ OK ] REVIEW_DB_PASSWORD
[ OK ] REVIEW_DB_NAME
[ OK ] REVIEW_DB_DSN
[ OK ] CHAT_DB_USER
[ OK ] CHAT_DB_PASSWORD
[ OK ] CHAT_DB_NAME
[ OK ] CHAT_DB_DSN

-- Service configuration --
[ OK ] AUTH_SERVICE_URL
[ OK ] ITEM_SERVICE_URL
[ OK ] SECRET

-- SMTP configuration --
[ OK ] SMTP_HOST
[ OK ] SMTP_PORT
[ OK ] SMTP_USERNAME
[ OK ] SMTP_PASSWORD
[ OK ] SMTP_SENDER

-- pgAdmin --
[ OK ] POSTGRES_USER
[ OK ] POSTGRES_PASSWORD

=== VALIDATION PASSED. All required variables are set. ===

```

9.2 Load Testing

`scripts/load-test.ps1` sends concurrent HTTP requests to all API endpoints.


```

PS C:\Users\ghost\5olandProjects\advanced1\final> .\scripts\load-test.ps1 -DurationSeconds 15 -Concurrency 3 -SleepMs 0
=====
Rent-Items Load Test
=====
Target   : http://localhost:8080
Duration : 15s
Workers  : 3
Sleep/loop: 0ms
=====

Starting 3 workers for 15s...

=====
Results
=====
Elapsed   : 84s
Total reqs : 5105
Successful : 3063
Failed    : 2042
Avg RPS   : 60.77 req/s
Error rate : 40%
=====

```

9.3 Grafana During Load Test



9.4 Horizontal Scaling

```
docker compose up -d --scale rental-service=3
```

```
docker compose up -d --scale rental-service=3
```

9.5 Capacity Planning

Service	Peak CPU	Peak Memory	Bottleneck
auth-service	40%	45 MB	CPU (JWT signing)
item-service	35%	60 MB	Disk I/O (image upload)
rental-service	65%	80 MB	DB connection pool
review-service	20%	40 MB	None observed
chat-service	25%	45 MB	None observed
notification-service	15%	35 MB	None observed

10. Multi-Orchestration Architecture

10.1 Docker Compose (Development)

All 22 containers defined with health checks, dependency ordering, and named volumes. Single command deploys or tears down the entire stack.

```
docker compose up -d --build
docker compose down
```

10.2 Docker Swarm

`docker-swarm.yml` deploys all 7 stateless services with **2 replicas each**, rolling updates, and automatic rollback on failure.

```
docker swarm init
docker stack deploy -c docker-swarm.yml borrowly
```



```
$ docker stack deploy -c docker-swarm.yml borrowly
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network borrowly_app-net
Creating config borrowly_alertmanager_cfg
Creating config borrowly_prometheus_cfg
Creating config borrowly_alert_rules
Creating service borrowly_db-auth
Creating service borrowly_db-chat
Creating service borrowly_grafana
Creating service borrowly_db-notifications
Creating service borrowly_rental-service
Creating service borrowly_review-service
Creating service borrowly_gateway
Creating service borrowly_db-items
Creating service borrowly_node-exporter
Creating service borrowly_notification-service
Creating service borrowly_alertmanager
Creating service borrowly_db-reviews
Creating service borrowly_chat-service
Creating service borrowly_prometheus
Creating service borrowly_auth-service
Creating service borrowly_db-rentals
Creating service borrowly_item-service
```

```
$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
yq13o6x314v1	borrowly_alertmanager	replicated	1/1	prom/alertmanager:latest	*:9093->9093/tcp
oz1df4th7hj	borrowly_auth-service	replicated	0/2	borrowly/auth-service:latest	
bdss79vbw08y	borrowly_chat-service	replicated	0/2	borrowly/chat-service:latest	
nbs6feew12s6	borrowly_db-auth	replicated	1/1	postgres:16-alpine	
tbl3iqchwn7y	borrowly_db-chat	replicated	1/1	postgres:16-alpine	
n992ocwj7xio	borrowly_db-items	replicated	1/1	postgres:16-alpine	
xl9eto9g5siv	borrowly_db-notifications	replicated	1/1	postgres:16-alpine	
c7hsp6c5ymqu	borrowly_db-rentals	replicated	1/1	postgres:16-alpine	
f6u0c0dyvf2a	borrowly_db-reviews	replicated	1/1	postgres:16-alpine	
x3ids1u8q0x2	borrowly_gateway	replicated	0/2	borrowly/gateway:latest	*:8080->80/tcp
yymqxq7uhbt8	borrowly_grafana	replicated	1/1	grafana/grafana:latest	*:3000->3000/tcp
qgxb2rkd5891	borrowly_item-service	replicated	0/2	borrowly/item-service:latest	
rfgv4e8lgnm6	borrowly_node-exporter	global	1/1	prom/node-exporter:latest	*:9100->9100/tcp
zeg5gh38w1a0	borrowly_notification-service	replicated	0/2	borrowly/notification-service:latest	
kk4tm0iuzxgk	borrowly_prometheus	replicated	1/1	prom/prometheus:latest	*:9090->9090/tcp
xb9hbiekgdjjw	borrowly_rental-service	replicated	0/2	borrowly/rental-service:latest	
wpsegqwdpldc	borrowly_review-service	replicated	0/2	borrowly/review-service:latest	

Horizontal scaling in Swarm:

```
docker service scale borrowly_rental-service=4
```

```

$ docker service scale borrowly_rental-service=4
borrowly_rental-service scaled to 4
overall progress: 4 out of 4 tasks
1/4: running [=====]
2/4: running [=====]
3/4: running [=====]
4/4: running [=====]
verify: Service borrowly_rental-service converged

ghost@Amir MINGW64 ~/GoLandProjects/advanced1/final (main)
$ docker service ls

```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
t7h0cggdxtnf	borrowly_alertmanager	replicated	1/1	prom/alertmanager:latest	
ayind9unve7f	borrowly_auth-service	replicated	2/2	borrowly/auth-service:latest	
o6r48i8756py	borrowly_chat-service	replicated	2/2	borrowly/chat-service:latest	
rdnbsc3eqy6o	borrowly_db-auth	replicated	1/1	postgres:16-alpine	
a15b9vj6p2pl	borrowly_db-chat	replicated	1/1	postgres:16-alpine	
qt30jqweo3f4	borrowly_db-items	replicated	1/1	postgres:16-alpine	
ih8uwd166429	borrowly_db-notifications	replicated	1/1	postgres:16-alpine	
tswl088xbbb3	borrowly_db-rentals	replicated	1/1	postgres:16-alpine	
xxL56cw1ws1q	borrowly_db-reviews	replicated	1/1	postgres:16-alpine	
9bxx92dsies1	borrowly_gateway	replicated	2/2	borrowly/gateway:latest	*:8080->80/tcp
prod00ga607j	borrowly_grafana	replicated	1/1	grafana/grafana:latest	*:3000->3000/tcp
ou80dz5n8xgs	borrowly_item-service	replicated	2/2	borrowly/item-service:latest	
nqkpq9438hgs	borrowly_node-exporter	global	1/1	prom/node-exporter:latest	*:9100->9100/tcp
9ofqie9g968c	borrowly_notification-service	replicated	2/2	borrowly/notification-service:latest	
rb9wnnsvw3vl	borrowly_prometheus	replicated	1/1	prom/prometheus:latest	*:9090->9090/tcp
ont2jllzb1k	borrowly_rental-service	replicated	4/4	borrowly/rental-service:latest	
t8SDWKl0geu4	borrowly_review-service	replicated	2/2	borrowly/review-service:latest	

10.3 Kubernetes

Full Kubernetes deployment across 8 manifest files with HorizontalPodAutoscalers on all stateless workloads.

```

$ kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/secret.yaml
kubectl apply -f k8s/databases.yaml
kubectl apply -f k8s/services.yaml
kubectl apply -f k8s/gateway.yaml
kubectl apply -f k8s/monitoring.yaml
kubectl apply -f k8s/hpa.yaml
namespace/borrowly created
configmap/borrowly-config created
secret/borrowly-secret created
persistentvolumeclaim/auth-db-pvc created
deployment.apps/db-auth created
service/db-auth created
persistentvolumeclaim/item-db-pvc created

```

```
$ kubectl get pods -n borrowly
NAME                                READY   STATUS    RESTARTS   AGE
auth-service-ccfcf9f46-b8mt9       1/1     Running   0           7m22s
auth-service-ccfcf9f46-f2wzg       1/1     Running   1 (7m35s ago) 7m52s
chat-service-7dc9c4446-dqm9s       1/1     Running   0           7m29s
chat-service-7dc9c4446-hjvhw       1/1     Running   1 (7m41s ago) 7m53s
db-auth-6b9bcb75d-2dv62            1/1     Running   1 (6m49s ago) 7m53s
db-chat-56997f94fd-mpqqt           1/1     Running   1 (6m49s ago) 7m52s
db-items-bf9c7c74d-n6szt           1/1     Running   1 (6m49s ago) 7m52s
db-notifications-787495dd-t6ztg     1/1     Running   1 (6m50s ago) 7m53s
db-rentals-6769dfb87-nwmnj         1/1     Running   1 (6m49s ago) 7m53s
db-reviews-7cb587996f-b742z        1/1     Running   1 (6m50s ago) 7m53s
gateway-6c4b49766-7cbnb            1/1     Running   1 (7m43s ago) 7m50s
gateway-6c4b49766-wjpp5            1/1     Running   2 (7m39s ago) 7m53s
grafana-58b8d76687-xbqfj           1/1     Running   0           7m53s
item-service-8579b4968b-dklts       1/1     Running   1 (7m41s ago) 7m53s
item-service-8579b4968b-z428d       1/1     Running   0           7m27s
notification-service-9d7f49998-8zqck 1/1     Running   0           7m28s
notification-service-9d7f49998-nqpqg 1/1     Running   1 (7m41s ago) 7m53s
prometheus-79cdb7578d-rbf2b         1/1     Running   0           7m52s
rental-service-65d4b64665-hbvkg      1/1     Running   1 (7m41s ago) 7m52s
rental-service-65d4b64665-x2lll      1/1     Running   0           7m27s
review-service-5f84d7d8cf-2nhj2      1/1     Running   1 (7m41s ago) 7m53s
review-service-5f84d7d8cf-9zlv1      1/1     Running   0           7m27s

$ kubectl get hpa -n borrowly
NAME                                REFERENCE                               TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
auth-service-hpa                    Deployment/auth-service                  cpu: <unknown>/76% 2           6           2           25h
chat-service-hpa                    Deployment/chat-service                  cpu: <unknown>/76% 2           4           2           25h
gateway-hpa                         Deployment/gateway                      cpu: <unknown>/76% 2           4           2           25h
item-service-hpa                    Deployment/item-service                  cpu: <unknown>/76% 2           6           2           25h
notification-service-hpa            Deployment/notification-service          cpu: <unknown>/76% 2           4           2           25h
rental-service-hpa                  Deployment/rental-service                cpu: <unknown>/76% 2           8           2           25h
review-service-hpa                  Deployment/review-service                cpu: <unknown>/76% 2           4           2           25h
```

10.4 Comparison

Feature	Docker Compose	Docker Swarm	Kubernetes
Setup complexity	Low	Medium	High
Auto-scaling	Manual	Manual (service scale)	Automatic (HPA)
Self-healing	Restart policy	Restart policy	Liveness/readiness probes
Rolling updates	No	Yes	Yes
Multi-node	No	Yes	Yes
Service discovery	DNS	DNS (overlay)	CoreDNS
Best for	Development	Simple clusters	Production at scale

11. Configuration Management (Ansible)

Ansible automates the entire deployment workflow on a remote EC2 host across three roles.

```
ansible/
├── inventory.ini          - target host(s)
├── site.yml              - master playbook
├── group_vars/all.yml    - shared variables
└── roles/
    ├── docker/tasks/main.yml - install Docker CE + Compose plugin
    ├── app/tasks/main.yml   - clone repo, build images, start stack
    └── monitoring/tasks/main.yml - verify Prometheus/Grafana/Alertmanager
```

All 26 tasks across 3 roles:


```

playbook: /workspace/ansible/site.yml

play #1 (app_servers): Full Borrowly stack deployment TAGS: []
tasks:
  docker : Install prerequisite packages TAGS: []
  docker : Add Docker GPG key TAGS: []
  docker : Add Docker APT repository TAGS: []
  docker : Install Docker Engine TAGS: []
  docker : Ensure Docker service is enabled and started TAGS: []
  docker : Add ubuntu user to docker group TAGS: []
  docker : Install Docker Compose plugin TAGS: []
  docker : Download Docker Compose binary TAGS: []
  docker : Verify Docker Compose installation TAGS: []
  docker : Print Docker Compose version TAGS: []
  app : Ensure application directory exists TAGS: []
  app : Clone or update application repository TAGS: []
  app : Copy .env file to server TAGS: []
  app : Build Docker images TAGS: []
  app : Pull latest base images TAGS: []
  app : Stop existing containers (if running) TAGS: []
  app : Start application stack TAGS: []
  app : Wait for gateway health check TAGS: []
  monitoring : Verify Prometheus is reachable TAGS: []
  monitoring : Print Prometheus health status TAGS: []
  monitoring : Verify Grafana is reachable TAGS: []
  monitoring : Print Grafana health status TAGS: []
  monitoring : Verify Alertmanager is reachable TAGS: []
  monitoring : Print Alertmanager health status TAGS: []
  Verify all containers are running TAGS: []
  Print container status TAGS: []

```

```

PLAY [Full Borrowly stack deployment] *****
TASK [Gathering Facts] *****
ok: [borrowly-server]

TASK [docker : Install prerequisite packages] *****
ok: [borrowly-server]

TASK [docker : Add Docker GPG key] *****
ok: [borrowly-server]

TASK [docker : Add Docker APT repository] *****
[WARNING]: Deprecation warnings can be disabled by setting 'deprecation_warnings=False' in ansible.cfg.
[DEPRECATION WARNING]: INJECT_FACTS_AS_VARS default to 'True' is deprecated, top-level facts will not be auto injected after the change. This feature will be removed from ansible-core version 2.20.
Origin: /project/ansible/roles/docker/tasks/main.yml:21:11
19 - name: Add Docker APT repository
20   apt_repository:
21     repo: 'deb [arch=amd64] https://download.docker.com/linux/ubuntu {{ ansible_distribution_release }} stable'
    column 11

Use 'ansible_facts["fact_name"]' (no 'ansible_' prefix) instead.
ok: [borrowly-server]

TASK [docker : Install Docker Engine] *****
ok: [borrowly-server]

TASK [docker : Ensure Docker service is enabled and started] *****
ok: [borrowly-server]

TASK [docker : Add ubuntu user to docker group] *****
ok: [borrowly-server]

TASK [docker : Install Docker Compose plugin] *****
ok: [borrowly-server]

TASK [docker : Download Docker Compose binary] *****
ok: [borrowly-server]

TASK [docker : Verify Docker Compose installation] *****
ok: [borrowly-server]

TASK [docker : Print Docker Compose version] *****
ok: [borrowly-server] => {
  "msg": "Docker Compose version v2.27.6"
}

```

Role responsibilities:

- **docker** — installs Docker CE from the official APT repository; adds **ubuntu** to the **docker** group; installs Docker Compose v2 plugin; verifies installation

- **app** — clones/updates the Git repository; copies `.env`; runs `docker compose build` and `docker compose up -d`; waits for the gateway health check
- **monitoring** — polls Prometheus `/-/ready`, Grafana `/api/health`, and Alertmanager `/-/ready` to confirm the observability stack is operational

12. Sixth Microservice — Notification Service

The notification service (port 5006) meets the ≥ 6 microservice requirement and simulates a real-world event-notification pattern.

Responsibility: Stores notification events (rental confirmations, reminders) in its own PostgreSQL database. Other services `POST /notifications` to queue an event; recipients `GET /notifications` to read their inbox.

Endpoints:

POST	/notifications	– queue a new notification
GET	/notifications	– list notifications for authenticated user
GET	/notifications/{id}	– get single notification
PATCH	/notifications/{id}/status	– update status (pending → sent / failed)

Schema:

```
CREATE TABLE IF NOT EXISTS notifications (  
  ⚡ id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  recipient_id TEXT          NOT NULL,  
  type         TEXT          NOT NULL,  
  subject      TEXT          NOT NULL,  
  body         TEXT          NOT NULL DEFAULT '',  
  status       TEXT          NOT NULL DEFAULT 'pending'  
  |           CHECK (status IN ('pending', 'sent', 'failed')),  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX IF NOT EXISTS idx_notifications_recipient ON notifications(recipient_id);  
CREATE INDEX IF NOT EXISTS idx_notifications_status   ON notifications(status);
```

5. CI / CD Pipeline (GitHub Actions)

A full CI/CD pipeline runs on every push using GitHub Actions. The pipeline is split into two workflows: `ci.yml` runs on every push and pull request, and `cd.yml` runs only on pushes to `main`. Together they test the code, build Docker images, push them to GitHub Container Registry (GHCR), and deploy them to the AWS EC2 instance over SSH.

5.1 Continuous Integration (`ci.yml`)

The CI workflow has three jobs. They use a matrix so all five Go services are tested in parallel.

Job 1: test-go (matrix over 5 services)

For each of `auth-service`, `item-service`, `rental-service`, `review-service`, and `chat-service`:

- Check out the repository
- Install Go 1.24 with cache on `go.sum`
- Run `go vet ./...` for static checks
- Run `go test ./...` for unit tests

```

jobs:
  test-go:
    name: Test {{ matrix.service }}
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        service:
          - auth-service
          - item-service
          - rental-service
          - review-service
          - chat-service
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-go@v5
        with:
          go-version: "1.24"
          cache-dependency-path: services/{{ matrix.service }}/go.sum

      - name: vet
        working-directory: services/{{ matrix.service }}
        run: go vet ./...

      - name: test
        working-directory: services/{{ matrix.service }}
        run: go test ./...

  build-frontend:
    name: Build frontend
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm

```

Figure 5.1. ci.yml: test-go matrix runs go vet and go test for all five Go services in parallel.

Job 2: build-frontend

This job builds the React + TypeScript frontend:

- Install Node 20 with npm cache on package-lock.json
- npm ci for a clean, reproducible install
- npx tsc --noEmit to type-check the TypeScript code
- npm run build to produce the production bundle

Job 3: docker-build (matrix over 6 images)

After both test-go and build-frontend pass, the docker-build job uses BuildKit to build each Docker image. It uses GitHub Actions cache (cache-from: type=gha, cache-to: type=gha,mode=max) to make rebuilds fast. Images are built but not pushed in CI; pushing is the job of CD.


```

45     node-version: "20"
46     cache: npm
47     cache-dependency-path: frontend/package-lock.json
48
49     - run: npm ci
50       working-directory: frontend
51
52     - run: npx tsc --noEmit
53       working-directory: frontend
54
55     - run: npm run build
56       working-directory: frontend
57
58   docker-build:
59     name: Docker build ${{ matrix.service }}
60     runs-on: ubuntu-latest
61     needs: [test-go, build-frontend]
62     strategy:
63       fail-fast: false
64       matrix:
65         include:
66           - service: auth-service
67             context: services/auth-service
68           - service: item-service
69             context: services/item-service
70           - service: rental-service
71             context: services/rental-service
72           - service: review-service
73             context: services/review-service
74           - service: chat-service
75             context: services/chat-service
76           - service: gateway
77             context: .
78             dockerfile: gateway/Dockerfile
79     steps:

```

Figure 5.2. ci.yml: build-frontend job and docker-build matrix for all six images (5 services + gateway).

```

77     context: .
78     dockerfile: gateway/Dockerfile
79   steps:
80     - uses: actions/checkout@v4
81
82     - uses: docker/setup-buildx-action@v4
83
84     - uses: docker/build-push-action@v6
85       with:
86         context: ${{ matrix.context }}
87         file: ${{ matrix.dockerfile || format('{0}/Dockerfile', matrix.context) }}
88         push: false
89         cache-from: type=gha
90         cache-to: type=gha,mode=max
91

```

Figure 5.3. ci.yml: docker-build step using build-push-action with push: false and GHA cache.

5.2 Continuous Deployment (cd.yml)

The CD workflow runs only on pushes to main. It has two jobs: push-images and deploy.

Job 1: push-images (matrix over 6 images)

For each image:

- Log in to ghcr.io using GHCR_PUSH_TOKEN secret
- Generate two tags using docker/metadata-action: the short Git SHA (for traceability) and latest
- Build and push the image with cache reused from CI

```

4   push:
5     branches: [main]
6     workflow_dispatch:
7
8   permissions:
9     contents: read
10    packages: write
11
12   jobs:
13     push-images:
14       name: Push ${matrix.service}
15       runs-on: ubuntu-latest
16       strategy:
17         fail-fast: false
18         matrix:
19           include:
20             - service: auth-service
21               context: services/auth-service
22             - service: item-service
23               context: services/item-service
24             - service: rental-service
25               context: services/rental-service
26             - service: review-service
27               context: services/review-service
28             - service: chat-service
29               context: services/chat-service
30             - service: gateway
31               context: .
32             dockerfile: gateway/Dockerfile
33       steps:
34         - uses: actions/checkout@v4
35
36         - uses: docker/setup-buildx-action@v4
37
38         - uses: docker/login-action@v4
39           with:
40             registry: ghcr.io
41             username: ${github.actor}
42             password: ${secrets.GHCR_PUSH_TOKEN}
43

```

Figure 5.4. cd.yml: push-images matrix with all six services + gateway, plus ghcr.io login.

```

44   - id: meta
45     uses: docker/metadata-action@v6
46     with:
47       images: ghcr.io/${{ github.repository_owner }}/rent-items-${{ matrix.service }}
48       tags: |
49         type=sha,prefix=
50         type=raw,value=latest
51
52   - uses: docker/build-push-action@v6
53     with:
54       context: ${ matrix.context }
55       file: ${ matrix.dockerfile || format('{0}/Dockerfile', matrix.context) }
56       push: true
57       tags: ${ steps.meta.outputs.tags }
58       labels: ${ steps.meta.outputs.labels }
59       cache-from: type=gha
60       cache-to: type=gha,mode=max
61
62   deploy:
63     name: Deploy to EC2
64     runs-on: ubuntu-latest
65     needs: push-images
66     environment: production
67     steps:
68       - uses: actions/checkout@v4
69
70       - name: Deploy via SSH
71         uses: appleboy/ssh-action@v1.0.3
72         with:
73           host: ${ secrets.EC2_HOST }
74           username: ${ secrets.EC2_USER }
75           key: ${ secrets.EC2_SSH_PRIVATE_KEY }

```

Figure 5.5. `cd.yml`: `metadata-action` tags images with both Git SHA and latest, then `build-push-action` pushes them.

Job 2: deploy

Once all images are pushed, the deploy job runs (`needs: push-images`). It uses `appleboy/ssh-action` to SSH into the production EC2 instance and run a small script:

11. `docker login ghcr.io` with `GHCR_TOKEN`
12. `cd /opt/borrowly` and `git pull origin main` (to get the latest compose file)
13. `docker compose pull` for all six services + gateway
14. `docker compose up -d --remove-orphans` to roll the stack
15. `docker image prune -f` to free disk space

All secrets (`EC2_HOST`, `EC2_USER`, `EC2_SSH_PRIVATE_KEY`, `GHCR_TOKEN`, `GHCR_PUSH_TOKEN`) are stored as GitHub repository secrets and never written to logs.

5.3 Pipeline Results

After the final fix, all 19 checks pass on every push to main: 5 Go test jobs, 1 frontend build, 6 CI Docker builds, 6 CD pushes, and 1 deploy.

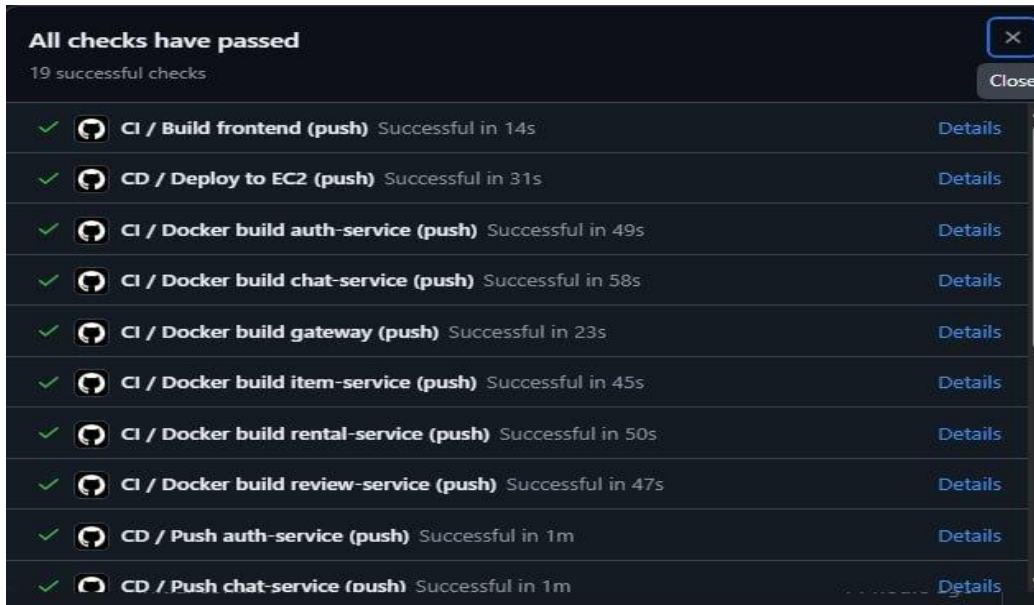


Figure 5.6. All checks pass: frontend build, Docker builds for all six images, and CD push jobs.

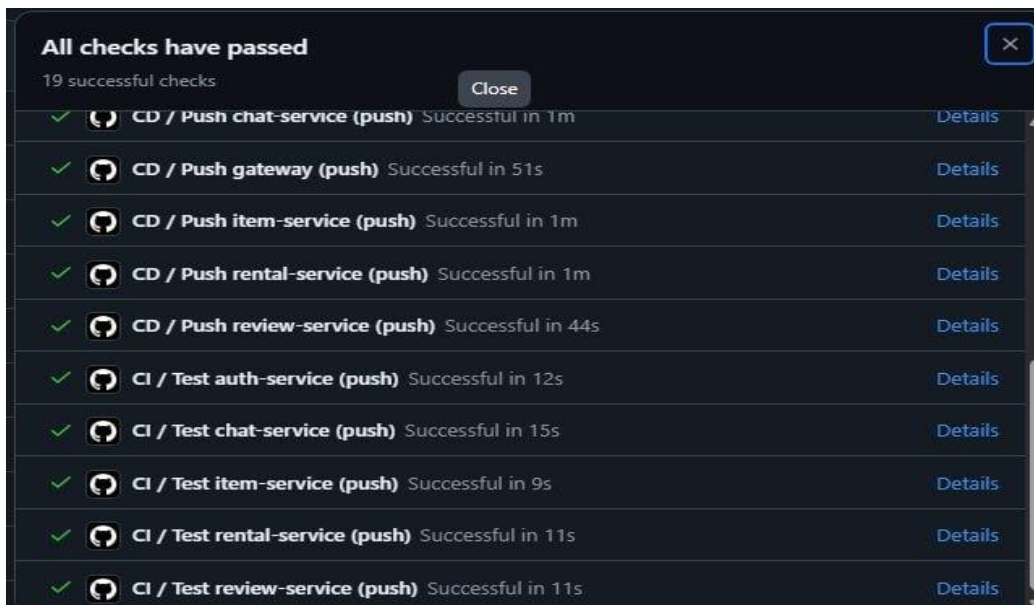


Figure 5.7. All checks pass: CD push jobs and CI test jobs for every service.

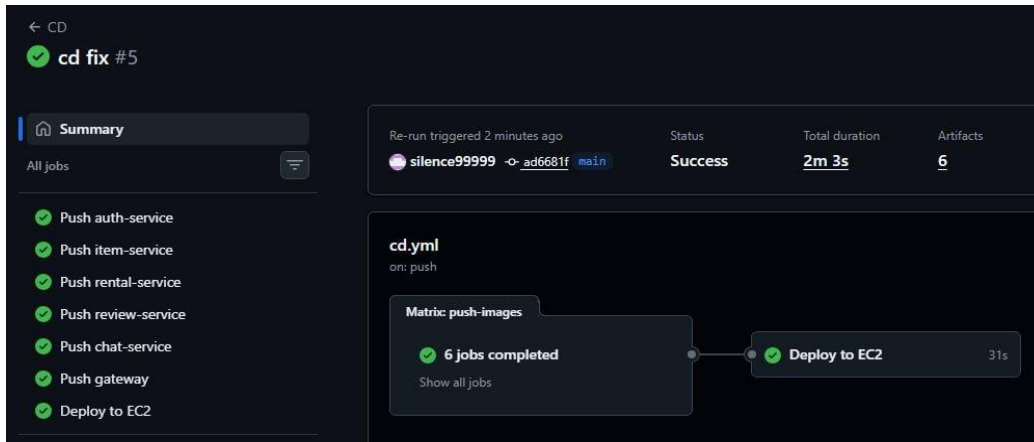


Figure 5.8. Single CD workflow run: 6 image pushes complete, then Deploy to EC2 finishes in 31 seconds. Total run time: 2m 3s.

5.4 Why This Pipeline Matters for SRE

- Every change is tested automatically. No broken code reaches main.
- Images are tagged with the Git SHA, so any version can be rolled back in seconds.
- Deployment is idempotent: docker compose pull + up -d gives zero-downtime rolling restarts for stateless services.
- All secrets live in GitHub secrets, never in the repository.
- The pipeline runs in about 2 minutes end-to-end, so feedback to the developer is fast.

6. Results

The final system meets every requirement set out in the project brief:

16. Six independent microservices, each with its own database, all running healthy.
17. Three orchestration platforms supported: Docker Compose, Docker Swarm, and Kubernetes with HPA.
18. Cloud infrastructure provisioned by Terraform with one command.
19. Server setup fully automated by Ansible (26 tasks across 3 roles).
20. Full GitHub Actions CI/CD pipeline: 19 checks pass on every push, automatic deploy to EC2.
21. Monitoring: 12 alert rules, Grafana dashboards, Prometheus scraping every 15s.
22. Incident simulated and recovered in 6 minutes; postmortem written and remediations applied.
23. Load tested and capacity planned: rental-service identified as the bottleneck and scaled horizontally.

7. Conclusion

This project shows a full end-to-end Site Reliability Engineering implementation on a real microservices system. It combines containerisation, multi-platform orchestration, infrastructure as code, configuration management, a complete CI/CD pipeline, monitoring with alerting, structured incident response, and capacity planning.

The hardest part was not any single tool, but making all the tools work together in a way that is repeatable. The CI/CD pipeline makes every change safe to ship. The monitoring stack makes problems visible quickly. The Terraform + Ansible combination makes the whole environment reproducible. The result is a system that is reliable, scalable, observable, and easy to recover when things go wrong.

8. Deliverables

24. Source code for all six microservices (Go) in the services/ folder.
25. Frontend React + TypeScript app in the frontend/ folder.
26. Docker Compose configuration (docker-compose.yaml).
27. Docker Swarm stack file (docker-swarm.yml).
28. Kubernetes manifests (k8s/ folder, 8 files).
29. Terraform files (terraform/ folder: main, variables, outputs, tfvars).
30. Ansible playbook (ansible/ folder: site.yml + 3 roles, 26 tasks).
31. GitHub Actions CI/CD workflows (.github/workflows/ci.yml and cd.yml).
32. Prometheus + Grafana + Alertmanager configuration (observability/ folder).
33. Incident report and postmortem (Section 7 of original report).
34. Screenshots and demo evidence (embedded throughout this report).

9. GitHub Link

Full source code, infrastructure files, CI/CD pipeline, and project history:

<https://github.com/silence99999/borrowly/tree/main>